Travel Itinerary Management System-Database Manual

DBS PROJECT REPORT

Submitted By:

Amina Asif (2024-CS-716)

Ahmad Ali (2024-CS-723)

Abiha Malik (2024-CS-727)

Submitted To:

Miss Rida Saeed

Department of Computer Science

University of Engineering and Technology,

Lahore, New Campus

Travel Itinerary Management System-Database Manual

Table of Contents

Table of Contents 2

1. Introduction 3

2. Database Schema Overview 3

3. Entity Descriptions 3

4. Relationships Between Entities 4

5. Keys and Constraints 5

6. ERD Diagram 6

ERD Explanation : 7

7. Normalization and Design Justification 7

Normalization in our database : 7

In 1st normal form: 8

In 2nd normal form: 9

In 3rd normal form: 12

8. Sample Visual Table Layouts 13

9. Views and Triggers 15

Views : 16

..... 17

Triggers : 17

11. SQL Script 19

12. Future Enhancements 19

13. Conclusion 20

Travel Itinerary Management System-Database Manual

1. Introduction

This manual provides a detailed explanation of the Travel Itinerary Management System database.
It includes schema descriptions, entity relationships, normalization strategies, and example visual layouts for better understanding.
The system is designed to manage travel bookings, packages, flights, accommodations, and related transactions efficiently.

2. Database Schema Overview

The database schema consists of 15 tables designed to capture various aspects of a travel itinerary.
The major tables include Customers, Destinations, Flights, Transportation, Guides, Packages, Bookings, Activities, Hotels, Payments, and more.
Each table has been normalized and connected through foreign key relationships to ensure data integrity.

3. Entity Descriptions

Below is a list of all tables with a brief description of each:

- **Customers**: Stores information about users booking the travel packages.
- **Destinations**: Contains destination details including city, country, and category.
- **Flights**: Stores flight information including airline, duration, and routes.
- **Transportation**: Holds ground transportation options, prices, and class types.
- **Guides**: Information about tour guides including contact and experience.

- **Packages**: Travel packages that link to destinations and include price, duration, and meal plans.
- **Bookings**: Links customers to packages and other services like flights and transportation.
- **Activities**: Available activities that can be associated with a booking.
- **Booking_Activities**: Many-to-many relationship table for bookings and activities.
- **Hotels**: Hotel information for accommodation during travel.
- **Hotel_Bookings**: Booking details for hotels linked to travel bookings.
- **Expenses**: Tracks any extra expenses related to a booking.
- **Payments**: Details of payment methods and amounts for each booking.
- **Cancellation_Refunds**: Stores cancellation reasons and refund details.

4. Relationships Between Entities

Entities in the database are connected through primary and foreign key relationships. Some of the main relationships include:

- Each booking is linked to one customer, one package, one flight, and one transportation option.

- Each package is related to one destination.

- Activities are linked to bookings via the Booking_Activities table.

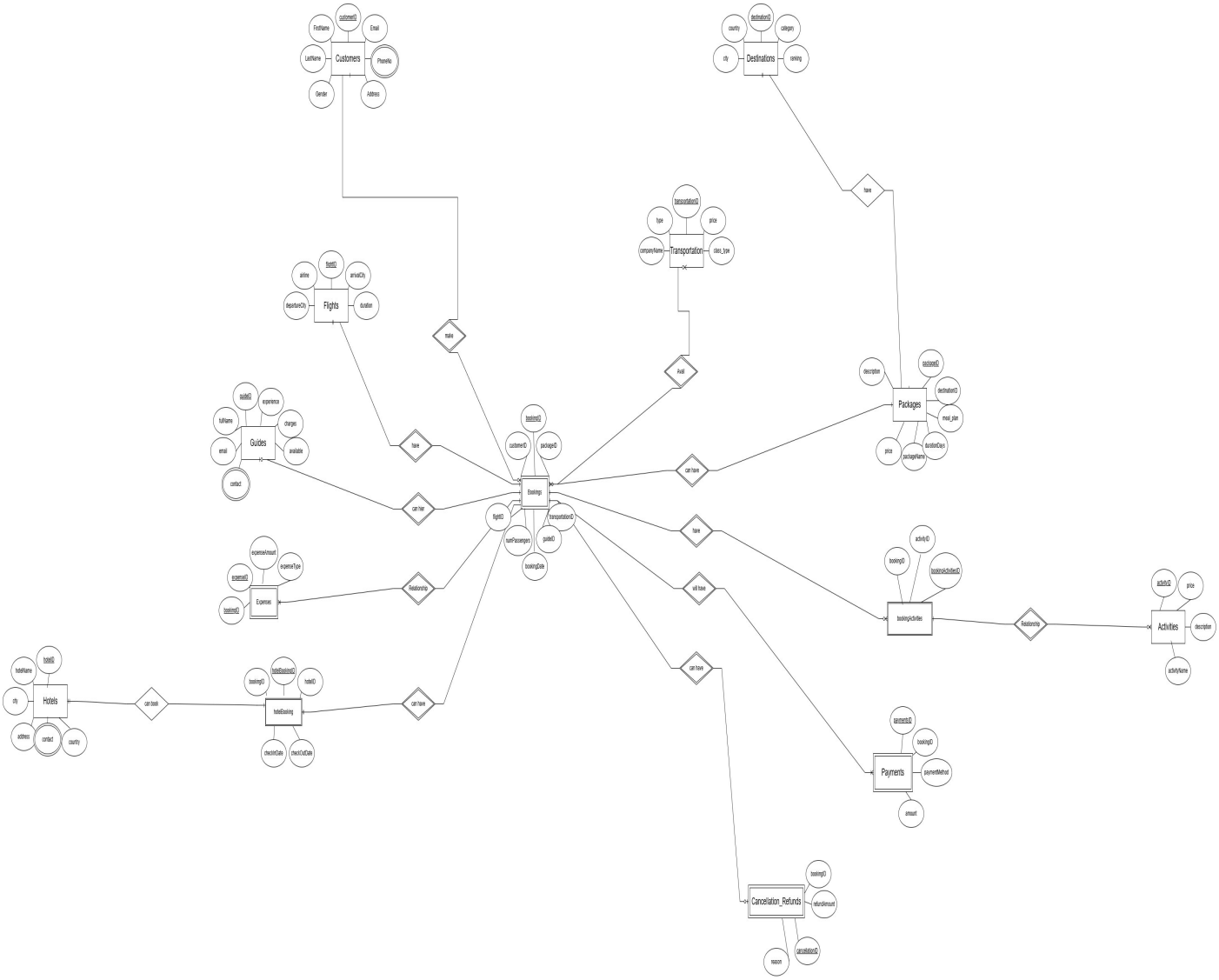
- Each booking can have one or more hotel bookings, payments, expenses, and optionally a guide.

5. Keys and Constraints

Each table has a primary key (mostly an IDENTITY field). Foreign keys are used to connect related tables.
CHECK constraints are used to ensure valid data entries (e.g., positive pricing, valid ratings).
UNIQUE constraints ensure no duplicate contact or email information is entered for customers, guides, or hotels.

Travel Itinerary Management System-Database Manual

6. ERD Diagram



ERD Explanation :

The ERD (Entity Relationship Diagram) shows all tables as entities and lines between them as relationships.
Primary keys are underlined, and foreign keys are indicated by connectors to their parent tables.
Some entities such as Booking_Activities act as bridge tables in many-to-many relationships.
Weak entities like Booking_Activities depend on other entities (e.g., bookings, activities) and do not have a strong primary key on their own.
Multi-valued and weak attributes (if any) are shown with double ovals or derived associations.

7. Normalization and Design Justification

The database is designed in at least 3rd Normal Form (3NF).
Redundancies are removed by dividing information into logical entities and connecting them through relationships.
This improves scalability, maintainability, and integrity of the data stored in the system.

Normalization in our database :

Before normalization:

Travel Itinerary Management System-Database Manual

```sql
CREATE TABLE BookingDetails (
    BookingID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    Phone VARCHAR(20),
    PackageName VARCHAR(100),
    Destination VARCHAR(100),
    FlightAirline VARCHAR(100),
    TransportType VARCHAR(50),
    GuideName VARCHAR(100),
    Activities VARCHAR(255),
    HotelName VARCHAR(100),
    PaymentAmount DECIMAL(10,2),
    Status VARCHAR(20)
);
```

In 1st normal form:

Activities table contains multiple value.

In 1st NF, there should be no multivalued attribute.

- Separate table for activities.
- Each activity gets its own row.

Travel Itinerary Management System-Database Manual

```sql
CREATE TABLE BookingDetails (
    BookingID INT,
    CustomerName VARCHAR(100),
    Phone VARCHAR(20),
    PackageName VARCHAR(100),
    Destination VARCHAR(100),
    FlightAirline VARCHAR(100),
    TransportType VARCHAR(50),
    GuideName VARCHAR(100),
    Activity VARCHAR(100),
    HotelName VARCHAR(100),
    PaymentAmount DECIMAL(10,2),
    Status VARCHAR(20)
);
```

In 2nd normal form:

In first normal form, Activities attribute is removed and a table is created Activities and then further bookingActivites. Cutomers, packages, flights, transportation, guides,hotels is added.

Primary key is BookingID.

Remove data that doesn't depend entirely on BookingID → like CustomerName, Phone.

Travel Itinerary Management System-Database Manual

```sql
CREATE TABLE customers (
    customerID INT IDENTITY(1,1) PRIMARY KEY,
    FirstName VARCHAR(100) NOT NULL,
    LastName VARCHAR(100) NOT NULL,
    Gender VARCHAR(25) NOT NULL,
    Email VARCHAR(255) UNIQUE,
    PhoneNo VARCHAR(20) UNIQUE,
    Address VARCHAR(255) NOT NULL
);
```

```sql
CREATE TABLE packages (
    packageID INT IDENTITY(1,1) PRIMARY KEY,
    packageName VARCHAR(100) NOT NULL,
    description VARCHAR(255) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    durationDays INT NOT NULL,
    meal_plan VARCHAR(20) NOT NULL,
    destinationID INT NOT NULL,
    CHECK (price > 0),
    CHECK (durationDays > 0)
);
```

```sql
CREATE TABLE flights (
    flightID INT IDENTITY(1,1) PRIMARY KEY,
    airline VARCHAR(100) UNIQUE,
    departureCity VARCHAR(100) NOT NULL,
    arrivalCity VARCHAR(100) NOT NULL,
    duration INT NOT NULL,
    CHECK (duration > 0),
);
```

```sql
CREATE TABLE transportation (
    transportationID INT IDENTITY(1,1) PRIMARY KEY,
    transportationType VARCHAR(100) UNIQUE,
    companyName VARCHAR(100) NOT NULL,
    price INT NOT NULL,
    class_type VARCHAR(20) NOT NULL,
    CHECK (price > 0)
);
```

Travel Itinerary Management System-Database Manual

```sql
CREATE TABLE guides (
    guideID INT IDENTITY(1,1) PRIMARY KEY,
    fullName VARCHAR(100) NOT NULL,
    email VARCHAR(255) UNIQUE,
    contactNumber VARCHAR(20) UNIQUE,
    experience INT NOT NULL,
    charges INT NOT NULL, -- TBN
    available VARCHAR(20) NOT NULL,
    CHECK (charges > 0)
);
```

```sql
CREATE TABLE hotels (
    hotelID INT IDENTITY(1,1) PRIMARY KEY,
    hotelName VARCHAR(100) NOT NULL,
    city VARCHAR(100) NOT NULL,
    address VARCHAR(255) NOT NULL,
    country VARCHAR(100) NOT NULL,
    contactNumber VARCHAR(20) UNIQUE
);
```

```sql
CREATE TABLE hotelBooking (
    hotelBookingID INT IDENTITY(1,1) PRIMARY KEY,
    bookingID INT NOT NULL,
    hotelID INT NOT NULL,
    checkInDate DATE NOT NULL,
    checkOutDate DATE NOT NULL,
    CHECK (checkInDate <= checkOutDate),
    FOREIGN KEY (bookingID) REFERENCES bookings(bookingID),
    FOREIGN KEY (hotelID) REFERENCES hotels(hotelID)
);
```

```sql
CREATE TABLE activities (
    activityID INT IDENTITY(1,1) PRIMARY KEY,
    activityName VARCHAR(100) UNIQUE,
    price DECIMAL(10,2) NOT NULL, -- TBN
    description VARCHAR(255) NOT NULL,
    CHECK (price > 0) -- TBN
);
```

```sql
CREATE TABLE bookingActivities (
    bookingActivitiesID INT IDENTITY(1,1) PRIMARY KEY,
    bookingID INT NOT NULL,
    activityID INT NOT NULL,
    FOREIGN KEY (bookingID) REFERENCES bookings(bookingID),
    FOREIGN KEY (activityID) REFERENCES activities(activityID)
);
```

In 3rd normal form:

There should be no transitive dependency in 3rd NF.

PackageName → Destination is transitive, so Destination should go in packages table.

```sql
CREATE TABLE bookings (
    bookingID INT IDENTITY(1,1) PRIMARY KEY,
    customerID INT NOT NULL,
    packageID INT NOT NULL,
    flightID INT NOT NULL,
    transportationID INT NOT NULL,
    numPassengers INT NOT NULL,
    bookingDate DATE NOT NULL,
    guideID INT,
    status VARCHAR(20) NOT NULL,
    FOREIGN KEY (customerID) REFERENCES customers(customerID),
    FOREIGN KEY (packageID) REFERENCES packages(packageID),
    FOREIGN KEY (flightID) REFERENCES flights(flightID),
    FOREIGN KEY (transportationID) REFERENCES transportation(transportationID),
    FOREIGN KEY (guideID) REFERENCES guides(guideID)
);
```

```sql
CREATE TABLE packages (
    packageID INT IDENTITY(1,1) PRIMARY KEY,
    packageName VARCHAR(100) NOT NULL,
    description VARCHAR(255) NOT NULL,
    price DECIMAL(10,2) NOT NULL,
    durationDays INT NOT NULL,
    meal_plan VARCHAR(20) NOT NULL,
    destinationID INT NOT NULL,
    CHECK (price > 0),
    CHECK (durationDays > 0)
);

ALTER TABLE packages
ADD CONSTRAINT FK_Packages_Destinations
FOREIGN KEY (destinationID) REFERENCES destinations(destinationID);
```

8. Sample Visual Table Layouts

Each table may be visually represented in a tabular format showing example records, headers, and relationships.
Tables like 'Bookings', 'Packages', and 'Customers' should include representative records to demonstrate how the schema works in practice.

Some of the visual tables of travel company database are shown below :

VISUAL RELATIONS :

Customers

Travel Itinerary Management System-Database Manual

customerID	FirstName	LastName	Email	PhoneNo	Address
1	Ali	Khan	ali.khan@example.com	03001234567	Lahore, Pakistan
2	Sara	Ahmed	sara.ahmed@example.com	03211234567	Karachi, Pakistan
3	Hassan	Raza	hassan.raza@example.com	03451234567	Islamabad, Pakistan
4	Zain	Iqbal	zain.iqbal@example.com	03007654321	Multan, Pakistan
5	Aisha	Khalid	aisha.khalid@example.com	03119876543	Faisalabad, Pakistan
6	Bilal	Javed	bilal.javed@example.com	03335678912	Rawalpindi, Pakistan
7	Noor	Fatima	noor.fatima@example.com	03456789012	Hyderabad, Pakistan
8	Omer	Saleem	omer.saleem@example.com	03012345678	Quetta, Pakistan
9	Maha	Hussain	maha.hussain@example.com	03211223344	Peshawar, Pakistan
10	Yasir	Ali	yasir.ali@example.com	03123456789	Sialkot, Pakistan

Destinations

destinationID	country	city	description
1	UAE	Dubai	A luxurious city with iconic modern architecture, sh...
2	Turkey	Istanbul	A city that straddles Europe and Asia, famous for i...
3	Malaysia	Kuala Lumpur	A bustling city with impressive skyscrapers, such a...
4	France	Paris	The city of lights, known for its romantic ambiance,...
5	USA	New York	A dynamic metropolis with landmarks like Times S...
6	UK	London	The capital of England, known for its historical lan...
7	Japan	Tokyo	A vibrant city blending traditional culture with mode...
8	Italy	Rome	The Eternal City, home to ancient history, the Colo...
9	Thailand	Bangkok	A bustling city known for its temples, street food, vi...
10	Germany	Berlin	A city known for its history, particularly around the ...

Bookings

bookingID	customerID	packageID	bookingDate	numPassengers	flightID	transportationID	tourID	guideID	budgetLimit
1	1	1	2023-10-05	2	1	1	1	5	1500.00
2	1	2	2024-05-01	1	3	2	2	2	1300.00
3	2	3	2024-07-01	3	1	1	3	3	2000.00
4	3	4	2024-08-10	1	2	3	2	1	2200.00
5	2	5	2024-04-15	2	5	1	1	2	2500.00
6	2	6	2023-12-12	1	2	4	2	3	1800.00
7	3	7	2024-09-20	4	3	1	3	4	3000.00
8	7	8	2023-11-05	2	1	2	3	1	2100.00
9	4	9	2024-06-14	3	3	3	4	2	1900.00
10	3	10	2023-10-30	1	5	5	4	3	1600.00

Payments

paymentID	bookingID	paymentDate	amount	paymentMethod
1	1	2023-10-01	1500.00	Credit Card
2	2	2024-04-25	1300.00	Bank Transfer
3	3	2024-06-25	2000.00	PayPal
4	4	2024-08-05	2200.00	Cash
5	5	2024-04-10	2500.00	Credit Card
6	6	2023-12-05	1800.00	Debit Card
7	7	2024-09-15	3000.00	Bank Transfer
8	8	2023-10-30	2100.00	PayPal
9	9	2024-06-10	1900.00	Cash
10	10	2023-10-25	1600.00	Credit Card

9. Views and Triggers

Views and Triggers are used in back-end of this project and details are as follows:

Travel Itinerary Management System-Database Manual

Views :

1. Customers who are on tour
2. See available guides
3. Which hotel was booked how many times
4. Packages expense only
5. Amount earned from transportation
6. Amount earned from activities
7. Amount earned from guides

Some of the views of our database are shown below:

- **View Available Guides:**

	guideID	fullName	email	contactNumber	experience	charges	available
1	1	John Smith	john.smith@example.com	0123456789	7	170	Yes
2	2	Sarah Johnson	sarah.johnson@example.com	0987654321	5	150	Yes
3	5	James Williams	james.williams@example.com	0194738291	3	120	Yes
4	6	Olivia Clark	olivia.clark@example.com	0182837465	4	130	Yes
5	8	Isabella Martinez	isabella.martinez@example.com	0146578290	3	120	Yes
6	10	Sophia Taylor	sophia.taylor@example.com	0192847365	5	150	Yes

- **View Transportation Expense:**

	transportationType	expenseAmount
1	Cruise	60.00
2	Private Car	50.00
3	Cruise	60.00
4	Private Car	50.00
5	Private Car	50.00
6	Taxi	30.00
7	Private Coach	50.00
8	Train	15.00
9	Bicycle	15.00
10	Bicycle	15.00
11	Cruise	60.00

- **View Package Expense:**

	packageName	expenseAmount
1	Island Getaway	1700.00
2	Beach Paradise	1200.00
3	Desert Oasis	1500.00
4	City Exploration	1300.00
5	City Exploration	1300.00
6	Safari Expedition	2200.00
7	Historical Journey	1400.00
8	Island Getaway	1700.00
9	Adventure Seekers	1800.00
10	Adventure Seekers	1800.00
11	Cultural Tour	1000.00

Triggers :

Backup Triggers :

Travel Itinerary Management System-Database Manual

1. Customers Table Trigger
2. Destinations Table Trigger
3. Flights Table Trigger
4. Transportation Table Trigger
5. Guides Table Trigger
6. Packages Table Trigger
7. Bookings Table Trigger
8. Destinations Table Trigger
9. Activities Table Trigger
10. Booking Activities Table Trigger
11. Hotels Table Trigger
12. Hotel Booking Table Trigger
13. Payments Table Trigger
14. Expenses Table Trigger
15. Cancellation Refunds Table Trigger

Other Triggers:

1. Package Expense Trigger

2. Transport Expense Trigger

3. Guide Expense Trigger

4. Activity Expense trigger

5. Set booking status cancelled when payment is refunded

6. Auto set booking status on confirmed when payment is paid

7. Check whether a Guide is available or not before choosing it

8. Check if refund_Amount is greater than total amount paid for the booking

11. SQL Script

The SQL script includes all CREATE TABLE statements with constraints, foreign keys, and checks.
It is used to initialize the database structure before populating it with real data.
Make sure to execute the script in a relational DBMS such as Microsoft SQL Server or MySQL with modifications if needed.

12. Future Enhancements

- Add user authentication and roles.
- Include travel insurance options.
- Enable package ratings and reviews.
- Create triggers for automatic updates.
- Expand cancellation policy tracking.

13. Conclusion

The Travel Itinerary Management System database has been carefully designed to ensure efficient management of all travel-related operations, from customer bookings to payments and accommodations.

By maintaining a normalized structure, enforcing strong entity relationships, and implementing necessary constraints, the system ensures data integrity, reduces redundancy, and supports scalability for future enhancements.

With the integration of views and triggers, the system not only safeguards critical information through backups but also automates important processes, enhancing overall reliability and performance.

This manual serves as a comprehensive guide for understanding, managing, and extending the database, ensuring that users and developers can effectively maintain and improve the system as business needs evolve.